

Algoritmos y Estructuras de Datos 3

Cuestionario del 2do. Examen Parcial

28 de Mayo de 2021, Periodo II - 2020

Observación 1. Todas las respuestas de este ejercitario tienen el objetivo de que el estudiante pueda aprender de sus respuestas. Por lo tanto, todas las respuestas están escritas en detalles y haciendo énfasis en la explicación. No se pretende que un estudiante durante un examen escriba con el detalle presentado en este escrito, pero, si que pueda expresar sus ideas de forma clara.

Observación 2. Este notebook fue elaborado con SageMath 9.2.

1. Grafos

Ejercicio 1.1

Sea $G_n = (V_n, E_n)$, con n entero positivo, un grafo construido de forma inductiva como sigue. El grafo G_1 es un triángulo. El grafo G_{n+1} se obtiene a partir de la unión disjunta de tres copias A_n, B_n, C_n del grafo G_n de acuerdo a la siguiente regla: El vértice en el tope de A_n coincide con el vértice en el fondo a la izquierda de B_n , y el vértice en el tope de C_n coincide con el vértice en el fondo a la derecha de B_n .

1. Dibuja G_2 y G_3 .
2. ¿Cuánto es el grado máximo de G_n ? Justifica tu respuesta.
3. ¿Cuánto es el grado mínimo de G_n ? Justifica tu respuesta.
4. ¿Cuánto es la cintura de G_n ? La cintura de un grafo es la longitud de su ciclo más corto. Justifica tu respuesta.
5. ¿Cuánto es el diámetro de G_n ? El diámetro D de un grafo $G = (V, E)$ se define como $D(G) = \max_{u,v \in V} d(u, v)$ donde $d(u, v)$ es la longitud del camino más corto entre u y v . Justifica tu respuesta.

Rúbrica del Ejercicio 1.1

Criterio	1 Punto	2 Puntos
Dibujo	Dibuja correctamente G_2 o G_3	Dibuja correctamente G_2 y G_3
Grados máximo	El grado máximo es correcto	El grado máximo es correcto y presenta una justificación correcta
Grados mínimo	El grado mínimo es correcto	El grado mínimo es correcto y presenta una justificación correcta
Cintura	La cintura es correcta	La cintura es correcta y presenta una justificación correcta

Criterio**1 Punto****2 Puntos**

Diámetro

El diámetro es correcto

El diámetro es correcto y presenta una justificación correcta

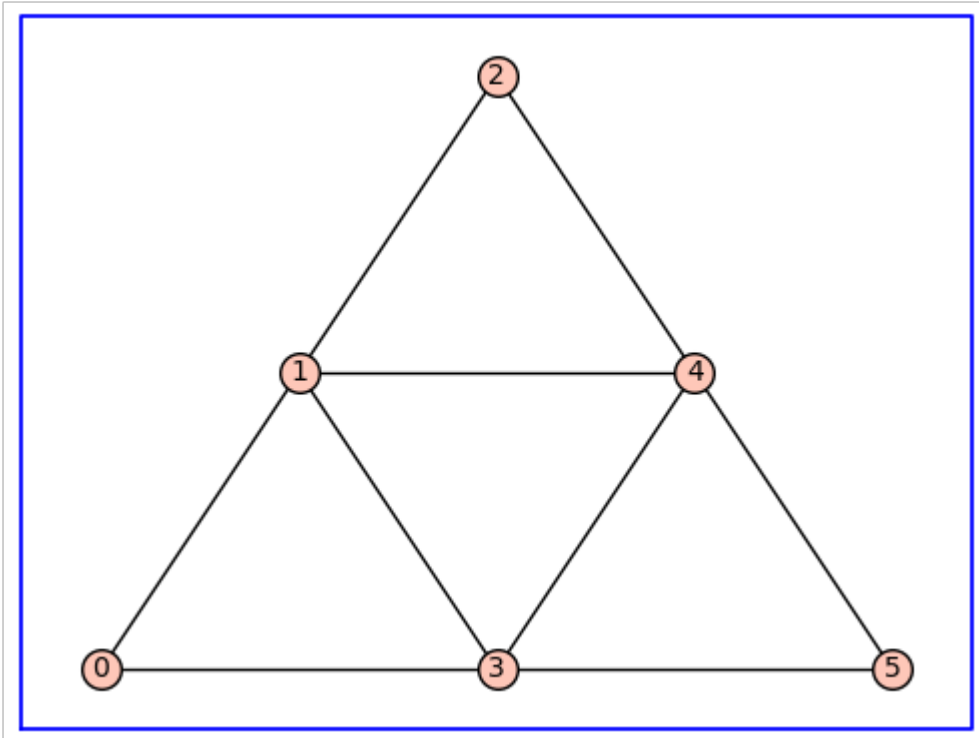
Solución del Ejercicio 1.1

El grafo solicitado en el ejercicio se llama *Grafo de Sierpinski-Gasket*.

El dibujo de G_2 se muestra abajo

```
In [93]: plot(graphs.SierpinskiGasketGraph(2), graph_border=True)
```

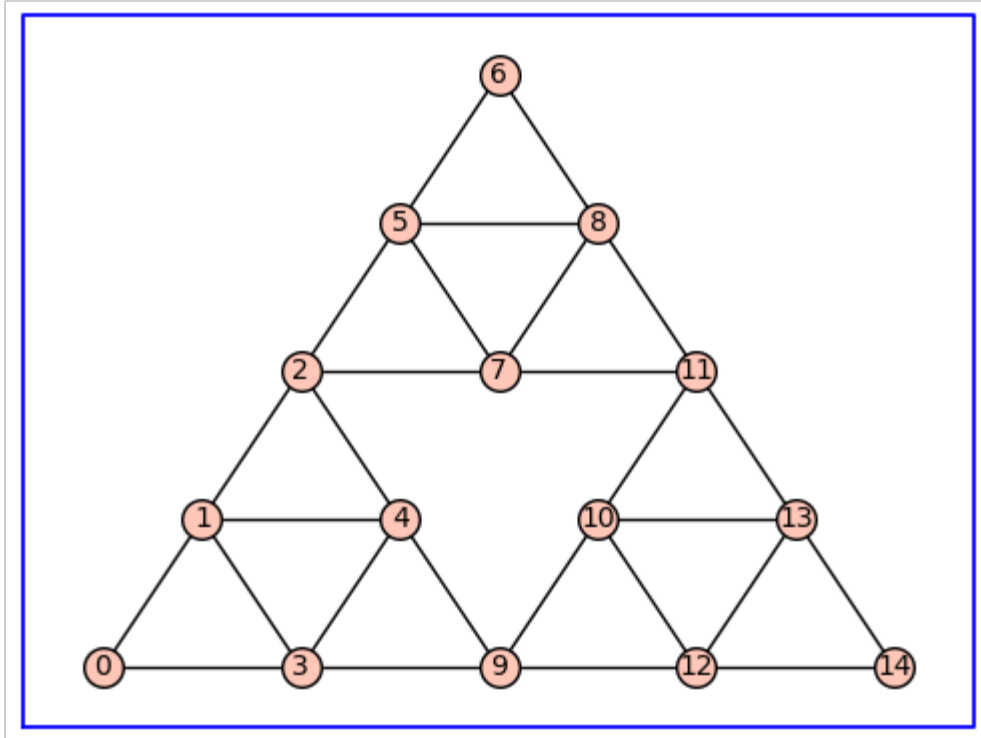
Out[93]:



El dibujo de G_3 se muestra abajo.

```
In [94]: plot(graphs.SierpinskiGasketGraph(3), graph_border=True)
```

Out[94]:



Como G_1 es un triángulo, entonces es un ciclo, y cada vértice es de grado 2. Sin embargo, para $n > 1$ debido a la construcción inductiva en donde los vértices en los toques coinciden tenemos que los vértices en los extremos de G_n tienen grado 2 y todos los otros vértices tienen grado 4. Los vértices de grado 4 son justamente los vértices que se utilizan para unir otras partes del grafo. En resumen, G_1 es un grafo regular de grado 2 y G_n , para $n > 1$, es un grafo con grado máximo 4 y grado mínimo 2.

Como la construcción es inductiva a partir de G_1 , el cual es un triángulo, tenemos que la cintura de G_n , para $n \geq 1$, es 3. Esto también se puede observar de las figuras de arriba.

Para contar el diámetro notemos que pasa durante la construcción inductiva de G_n . Para G_1 el diámetro es 1, y para G_2 el diámetro es 2. Luego para G_3 es 4, y para G_4 es 8. En cada paso se duplica el diámetro, y esto es porque G_n se construye a partir de G_{n-1} . Basta con observar la longitud del perímetro de la figura triangular que posee G_n . Entonces tenemos que el diámetro de G_n es 2^{n-1} .

Ejercicio 1.2

Sea $G_n = (V_n, E_n)$, con n entero positivo, un grafo en donde V_n es el conjunto de permutaciones del conjunto $\{1, 2, \dots, n\}$. Dos vértices están conectados por una arista si y solo si uno puede obtenerse a partir del otro intercambiando los elementos en las posiciones i e $i + 1$, para $1 \leq i \leq n - 1$.

1. Define formalmente G_n escribiendo ambos V_n y E_n por extensión o compresión.
2. Dibuja G_2 y G_3 .

3. ¿Cuánto es el grado máximo de G_n ? Justifica tu respuesta.
4. ¿Cuánto es el grado mínimo de G_n ? Justifica tu respuesta.
5. ¿Cuánto es el número total de aristas de G_n ? Justifica tu respuesta.

Rúbrica del Ejercicio 1.2

Criterio	1 Punto	2 Puntos
Definición formal	Define formalmente y correctamente el conjunto de vértices o aristas	Define formalmente y correctamente el conjunto de vértices y aristas
Dibujo	Dibuja correctamente G_2 o G_3	Dibuja correctamente G_2 y G_3
Grados máximo	El grado máximo es correcto	El grado máximo es correcto y presenta una justificación correcta
Grados mínimo	El grado mínimo es correcto	El grado mínimo es correcto y presenta una justificación correcta
Aristas	El número de aristas es corecto	El número de aristas es correcto y presenta una justificación correcta

Total 10 puntos

Solución del Ejercicio 1.2

Recuerda que una permutación π es un función inyectiva en el conjunto $\{1, 2, \dots, n\}$. Entonces podemos definir el conjunto de vértices

$$V_n = \{(\pi(1), \pi(2), \dots, \pi(n)) : \pi \text{ es un permutación en } \{1, 2, \dots, n\}\}.$$

Para definir el conjunto de aristas, consideremos dos vértices u y v y sus permutaciones correspondientes π_u y π_v , respectivamente. Entonces escribamos una relación $u \sim v$ si y solo si existe exactamente i tal que $\pi_v(i+1) = \pi_u(i)$, $\pi_v(i) = \pi_u(i+1)$, y para todo $i' \neq i$ e $i' \neq i+1$ se cumple que $\pi_u(i') = \pi_v(i')$. Así tenemos que $u \sim v$ significa que la permutación π_v puede obtenerse a partir de π_u intercambiando los elementos en las posiciones i e $i+1$. De esta forma el conjunto de aristas se define como

$$E_n = \{uv \in V \times V : u \sim v\}.$$

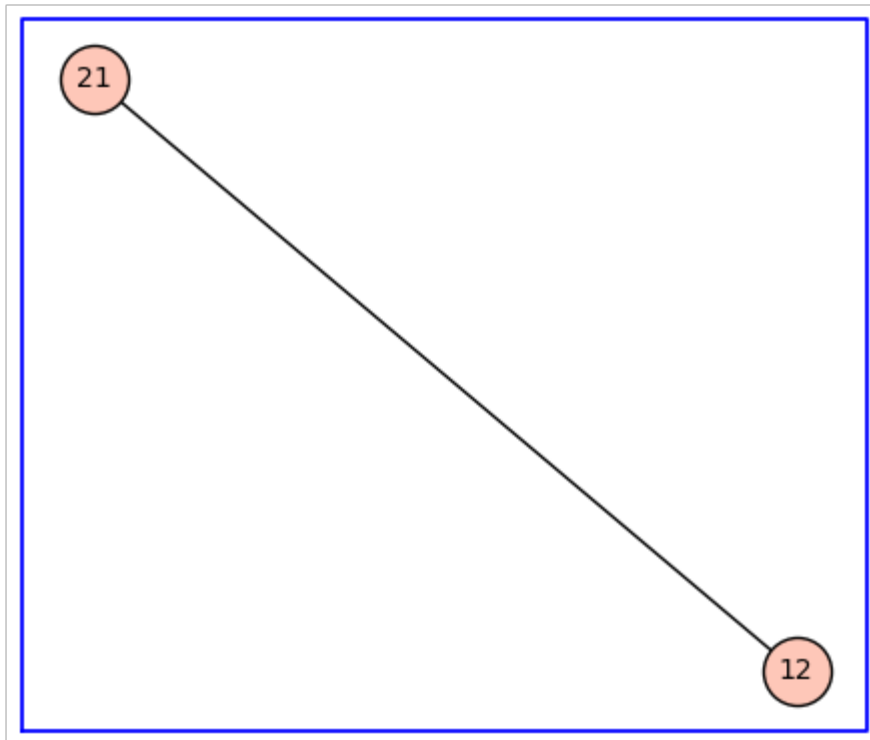
Este grafo recién definido se conoce como el grafo de *Bubblesort*, porque efectivametente el algoritmo *BubbleSort* es un recorrido sobre este grafo a través de sus operaciones de intercambio.

Las gráfica de G_2 y G_3 se muestran a abajo.

In [105...

```
from sage.graphs.generators.families import BubbleSortGraph
plot(BubbleSortGraph(2), vertex_size=600, graph_border=True)
```

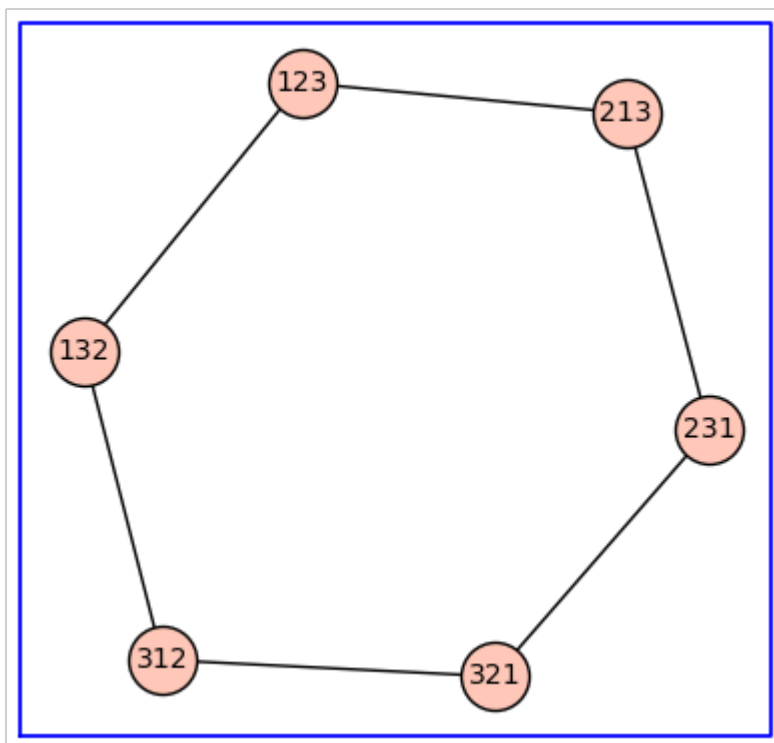
Out[105...



In [106...

```
plot(BubbleSortGraph(3), vertex_size=600, graph_border=True)
```

Out[106...



Los grados mínimo y máximo son iguales, porque cada vértice tiene exactamente $n - 1$ vecinos. Esto es porque se intercambian siempre los elementos $i = 1, 2, \dots, n - 1$. Esto hace que G_n sea un grafo regular.

Para contar el número de aristas primero notemos que hay $n!$ vértices. Esto es porque cada

vértice es una permutación en el conjunto $\{1, 2, \dots, n\}$. Luego, cada vértice tiene $n - 1$ vecinos. Por lo tanto en total tenemos $n! \cdot (n - 1)/2$ aristas.

Ejercicio 1.3

Sea $G_n = (V_n, E_n)$ un grafo, con $n > 1$ un número entero, donde cada vértice en V_n es un subconjunto de $\{1, 2, \dots, 2n - 1\}$ de tamaño $n - 1$. Dos vértices u y v están conectados por una arista si y solo si $u \cap v = \emptyset$.

1. Define formalmente G_n escribiendo ambos V_n y E_n por extensión o compresión.
2. Dibuja G_2 y G_3 .
3. ¿Cuánto es el grado máximo de G_n ? Justifica tu respuesta.
4. ¿Cuánto es el grado mínimo de G_n ? Justifica tu respuesta.
5. ¿Cuánto es el número total de aristas de G_n ? Justifica tu respuesta.

Rúbrica del Ejercicio 1.3

Criterio	1 Punto	2 Puntos
Definición formal	Define formalmente y correctamente el conjunto de vértices o aristas	Define formalmente y correctamente el conjunto de vértices y aristas
Dibujo	Dibuja correctamente G_2 o G_3	Dibuja correctamente G_2 y G_3
Grados máximo	El grado máximo es correcto	El grado máximo es correcto y presenta una justificación correcta
Grados mínimo	El grado mínimo es correcto	El grado mínimo es correcto y presenta una justificación correcta
Aristas	El número de aristas es corecto	El número de aristas es correcto y presenta una justificación correcta

Total 10 puntos

Solución del Ejercicio 1.3

Sea $[2n - 1] = \{1, 2, \dots, 2n - 1\}$. El conjunto de vértices se define como

$$V_n = \{S \subseteq [2n - 1] : |S| = n - 1\},$$

y el conjunto de aristas es

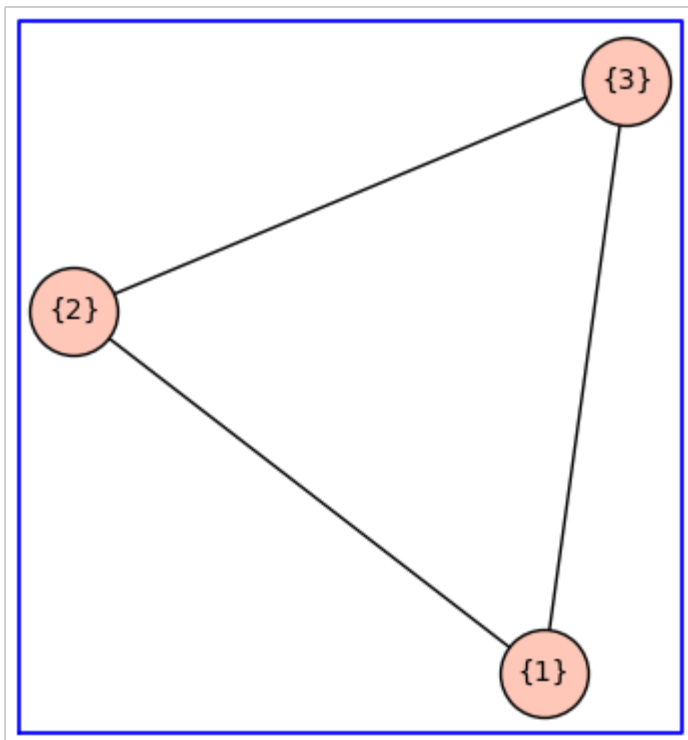
$$E_n = \{\{S, T\} : S \subseteq [2n - 1], T \subseteq [2n - 1], S \cap T = \emptyset\}.$$

El grafo solicitado en este ejercicio se llama *grafo impar*. Las gráficas de G_2 y G_3 se muestran abajo.

In [107...

```
plot(graphs.OddGraph(2), vertex_size=1000, graph_border=True)
```

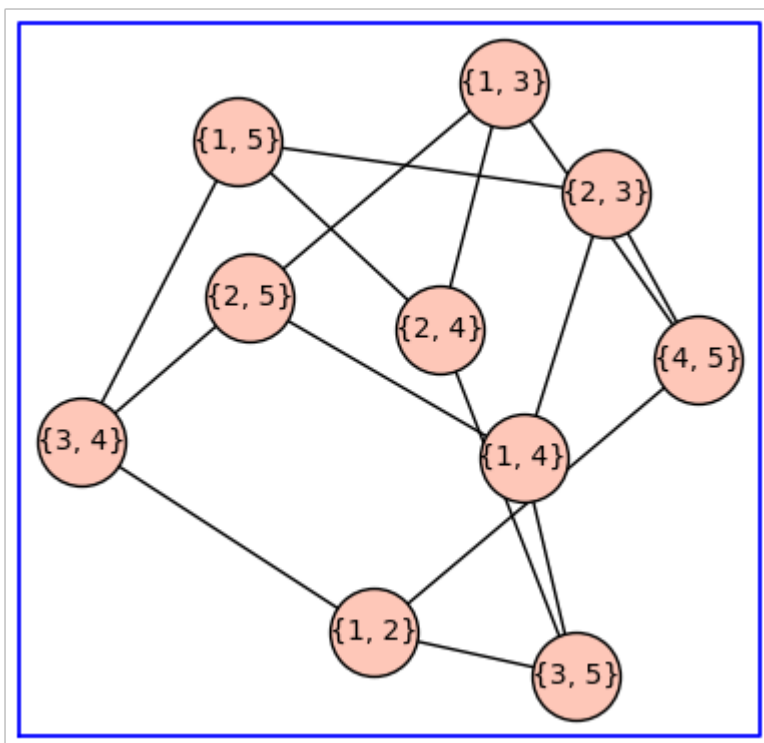
Out[107...



In [109...

```
plot(graphs.OddGraph(3), vertex_size=1000, graph_border=True)
```

Out[109...



El grafo G_n es un grafo regular de grado n . Para ver esto notemos los vértices son subconjuntos de tamaño $n - 1$ de un conjunto más grande $\{1, 2, n - 1, n, n + 1, \dots, 2n - 1\}$. Como los vecinos de un vértice v tienen que tener una intersección vacía con v y sabemos que $|v| = n - 1$, entonces hay n elementos que no pertenecen a v , porque $2n - 1 - (n - 1) = n$. A partir de

estos n elementos que no están en v podemos construir n conjuntos de tamaño $n - 1$ simplemente eligiendo un elemento por turno para no incluir. Por lo tanto v tiene n vértices adyacentes.

El número de vértices es $\binom{2n-1}{n-1}$ porque tomamos subconjuntos de tamaño $n - 1$, y por lo tanto, el número de aristas es $\binom{2n-1}{n-1} / 2$ donde n viene del grado de G .

Ejercicio 1.4

Sea $G_n = (V_n, E_n)$ un grafo, con n un entero positivo, donde n copias de un ciclo C_3 se unen en un vértice en común. Formalmente, sea $C_{3,i} = (V_{3,i}, E_{3,i})$ un ciclo de tamaño 3 con vértices en $\{0, 1_i, 2_i\}$ donde $1 \leq i \leq n$. Entonces $V_n = \cap_{i=1}^n V_{3,i}$ y $E_n = \cup_{i=1}^n E_{3,i}$.

1. Dibuja G_2 y G_3 .
2. ¿Cuánto es el grado máximo de G_n ? Justifica tu respuesta.
3. ¿Cuánto es el grado mínimo de G_n ? Justifica tu respuesta.
4. ¿Cuánto es el número de vértices? Justifica tu respuesta.
5. ¿Cuánto es el número de aristas? Justifica tu respuesta.
6. ¿Cuánto es la cintura de G_n ? La cintura de un grafo es la longitud de su ciclo más corto. Justifica tu respuesta.

Rúbrica del Ejercicio 1.4

Criterio	1 Punto	2 Puntos
Dibujo	Dibuja correctamente G_2 o G_3	Dibuja correctamente G_2 y G_3
Grados máximo	El grado máximo es correcto	El grado máximo es correcto y presenta una justificación correcta
Grados mínimo	El grado mínimo es correcto	El grado mínimo es correcto y presenta una justificación correcta
Vértices	El número de vértices es corecto	El número de vértices es correcto y presenta una justificación correcta
Aristas	El número de aristas es corecto	El número de aristas es correcto y presenta una justificación correcta
Cintura	La cintura es correcta	La cintura es correcta y presenta una justificación correcta

Total 12 puntos

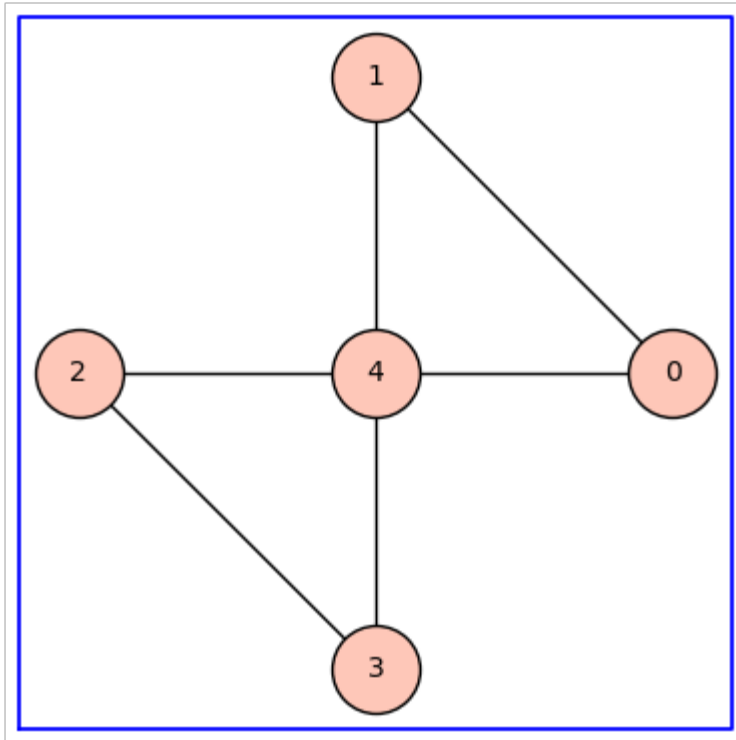
Solución del Ejercicio 1.4

El grafo descrito en este ejercicio se llama *grafo de la amistad*. Las gráficas de G_2 y G_3 se muestran abajo.

In [113...

```
plot(graphs.FriendshipGraph(2), vertex_size=1000, graph_border=True)
```

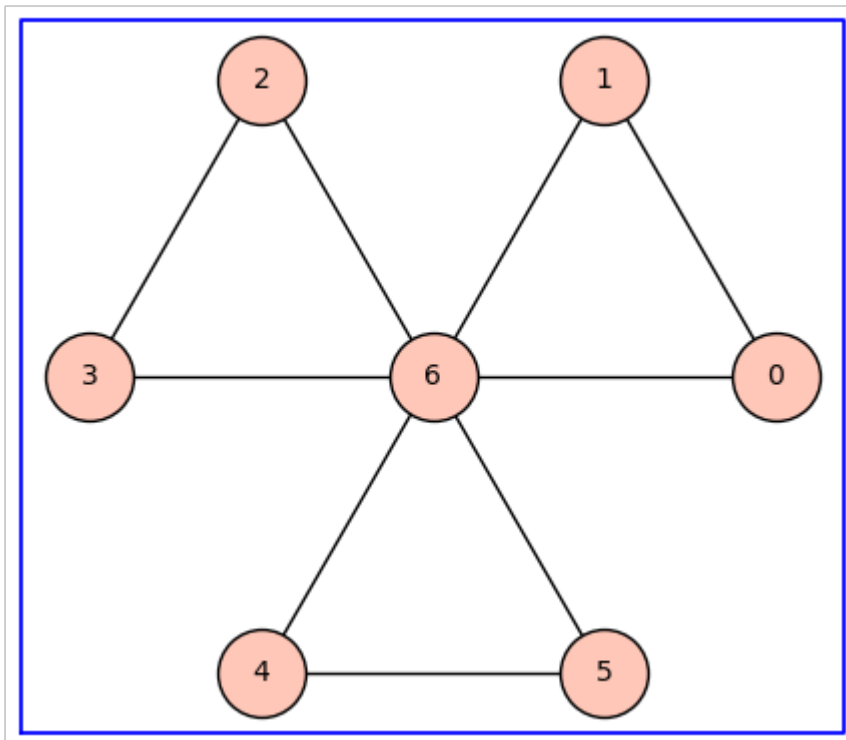

Out[113...



In [116...

```
plot(graphs.FriendshipGraph(3), vertex_size=1000, graph_border=True)
```

Out[116...



El grafo G_1 es un grafo regular de grado 2 y por lo tanto es un ciclo. Para $n > 1$ tenemos vértices de grado 2 que corresponden a los vértices que no tienen en común los ciclos C_3 , y hay exactamente un vértice de grado $2n$ que corresponden al vértice que es común a todos los ciclos C_3 .

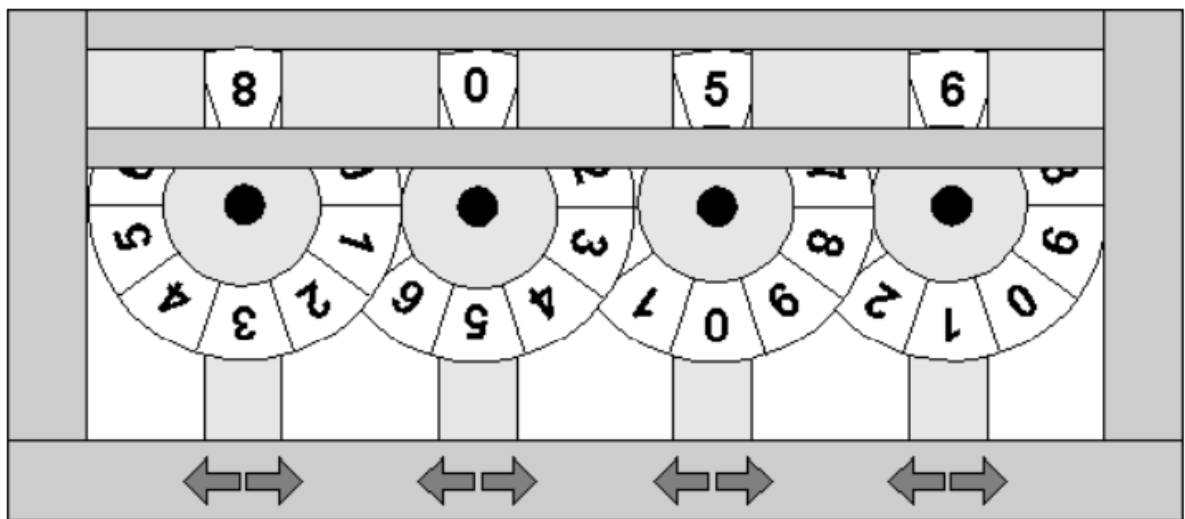
El número de vértices es $3n - n + 1 = 2n + 1$, porque tenemos 3 ciclos y como se intersectan en un vértice tenemos que restar n y sumar 1. El número de aristas es $3n$ porque tenemos n ciclos C_3 y no hay aristas en común.

La cintura es 3 porque el grafo se construye a partir de ciclos de tamaño 3.

2. Algoritmos de Grafos

Ejercicio 2.1

Considera las cuatro ruedas de combinación numérica que se muestran abajo en la figura. Cada rueda tiene 10 dígitos del 0 al 9 y pueden girar a la izquierda o a la derecha presionando botones en forma de flecha debajo de cada rueda. Si se presiona el botón de izquierda, la rueda que está directamente encima gira una posición en sentido de las manecillas del reloj. Si se presiona el botón de derecha, la rueda que está directamente encima gira una posición en sentido contrario a las manecillas del reloj.



En este problema tenemos una configuración inicial de 4 dígitos en las ruedas indicados por los dígitos en el tope de cada rueda. Supongamos que la configuración inicial es $A_1 A_2 A_3 A_4$, donde A_j es el dígito en el tope de la rueda j . También se da como entrada al problema una colección de configuraciones prohibidas $P_{i1} P_{i2} P_{i3} P_{i4}$, con $i = 1, 2, \dots, k$.

Escribe un algoritmo que toma como entrada la configuración inicial (A_1, A_2, A_3, A_4) , una configuración final (B_1, B_2, B_3, B_4) , una lista de k configuraciones prohibidas $(P_{i1}, P_{i2}, P_{i3}, P_{i4})$, y retorna el número mínimo de veces que hay que presionar los botones para llegar desde la configuración inicial a la configuración final sin pasar por una configuración prohibida.

1. Modelar el problema como un problema de grafos. Presenta una definición formal del grafo escribiendo por comprensión o extensión los conjuntos de vértices y aristas.
2. Dibuja un ejemplo pequeño de al menos 4 vértices y 6 aristas del grafo definido.

3. Escribe un algoritmo basado en BFS para resolver el problema solicitado. Presenta una explicación del funcionamiento de tu algoritmo.
4. ¿Cuál es la complejidad temporal de tu algoritmo si tuvieras n ruedas en lugar de 4? Escribe la complejidad utilizando notación asintótica en términos del número de vértices y aristas del grafo. Presenta también una justificación de esa complejidad.

Rúbrica del Ejercicio 2.1

Criterio	1 Punto	2 Puntos
Grafo	Define formalmente y correctamente el conjunto de vértices o aristas	Define formalmente el conjunto de vértices y aristas
Dibujo del grafo	Presenta un dibujo correcto basado en la definición formal	
Algoritmo	Presenta un pseudocódigo que retorna el resultado correcto en la mayoría de las entradas	Presenta un pseudocódigo que retorna el resultado correcto en todas las entradas
Explicación	Escribe una explicación del algoritmo presentado justificando su funcionamiento	
Complejidad	La complejidad asintótica es correcta	La complejidad asintótica es correcta y presenta una justificación correcta de la complejidad

Total 8 puntos

Solución del Ejercicio 2.1

Para escribir la definición formal del grafo, consideremos la lista de k configuraciones prohibidas $\{(P_{i1}, P_{i2}, P_{i3}, P_{i4})\}_{n \geq 1}$. Definimos el grafo $G = (V, E)$ del problema como el espacio de búsqueda de las configuraciones de la ruedas. El conjunto de vértices es

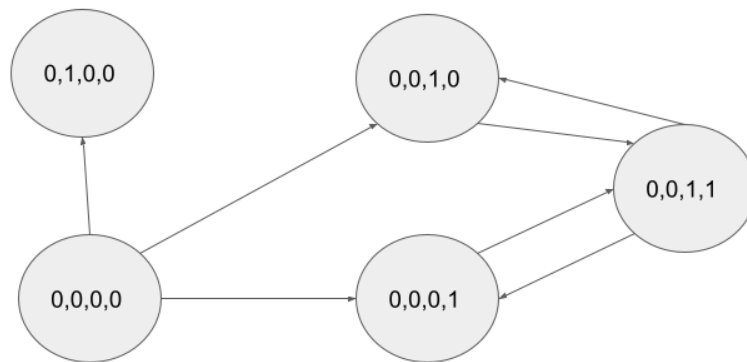
$$V = \{(C_1, C_2, C_3, C_4) \in \{0, \dots, 9\} : \forall i \{1, \dots, k\} \quad (C_1, C_2, C_3, C_4) \neq (P_{i1}, P_{i2}, P_{i3}, P_{i4})\}.$$

Para poder definir el conjunto de aristas es tenemos que definir la operación de los botones en forma de flecha. Sean dos configuracion $C = (C_1, C_2, C_3, C_4)$ y $D = (D_1, D_2, D_3, D_4)$.

Definimos la operacion $\xrightarrow{i}$ como $(C_1, C_2, C_3, C_4) \xrightarrow{i} (D_1, D_2, D_3, D_4)$ donde $D_j = C_j$ si $j \neq i$ y $D_j = C_j + 1$ si $j = i$. La operación $\xrightarrow{i}$ indica que giramos la rueda i en sentido contrario a las manecillas de reloj. Análogamente definimos la operación $\xleftarrow{i}$. Entonces podemos definir el conjunto de aristas como

$$E = \{(C, D) \in V \times V : \exists i \quad C \xleftarrow{i} D \vee C \xrightarrow{i} D\}.$$

Una parte del grafo definido se muestra abajo.



```

//A=[A1,A2,A3,A4] es la configuración inicial
//B=[B1,B2,B3,B4] es la configuración final
//P es una lista de k configuraciones prohibidas
COMBINACIONES(A=[A1,A2,A3,A4],A=[B1,B2,B3,B4],P)
begin
    Q = [A] //se inicializa la cola Q con la configuración
    inicial
    Inicializar un vector C de 10^4 elementos todos en 0 //este
    vector es el contador de operaciones
    C[A] = 0
    while Q no está vacío do
        D = Q.dequeue()
        if D = B then //si D es la configuración final,
        retornamos el valor de C[D]
            return C[D]
        for i=1 to 4 do
            R = RIGHT(D,i) //R es la configuración que se obtiene
            de girar la rueda i en D a la derecha una posición
            if R no es una configuración prohibida then
                Q.inject(R)
                C[R] = C[R]+1
            end-if
            L = LEFT(D,i) //L es la configuración que se obtiene
            de girar la rueda i en D a la izquierda una posición
            if L no es una configuración prohibida then
                Q.inject(L)
                C[L] = C[L]+1
            end-if
        end-for
    end-while
end

```

Este algoritmo lo que hace es recorrer el grafo definido a partir de la configuración inicial. En una configuración D , si D no es la configuración final, se generan todas las configuraciones que son alcanzables a partir de D a través de las operaciones RIGHT y LEFT. Las configuraciones alcanzables se insertan en la cola Q para su procesamiento posterior. El recorrido es en BFS

porque las configuraciones son visitadas utilizando la estructura de cola Q . Cada vez que una configuración es generada, se incrementa su contador correspondiente en el vector C . Esto también solo se hace si la configuración generada no es una configuración prohibida. Si en algún momento el elemento D que es extraído de la cola Q es la configuración final, se retorna su valor en el vector C .

Para analizar la complejidad debemos de conocer la cantidad de vértices y aristas del grafo de búsqueda. Como cada vértice es una configuración, tenemos entonces 10^n vértices exactamente. El número de aristas es más complejo porque depende de cuantas configuraciones prohibidas hay. Podemos, sin embargo, estimar el número de aristas con notación asintótica. Por cada configuración tenemos 8 configuraciones alcanzables debido a las operaciones RIGHT y LEFT. Por lo tanto, en el peor caso cuando no hay configuraciones prohibidas, hay a lo sumo 4×10^n aristas. Además, cada vez que se verifica si L o R es una configuración prohibida, eso requiere hacer k comparaciones entre vectores de longitud n , lo cual da un costo de $O(k \cdot n)$. Por lo tanto, la complejidad del algoritmo es $O(kn2^n)$ porque el recorrido es en BFS y se trata de un grafo de tamaño exponencial.

Ejercicio 2.2

Sea $C_1, C_2, \dots, C_n$ ciertos compuestos químicos diferentes donde $n > 2$. Escribimos $C_i + C_j$, con $i \neq j$, para indicar que los compuestos C_i y C_j se enlazan químicamente. Si en algún momento se enlazan 3 pares de compuestos utilizando solo 3 compuestos, entonces resulta en un explosivo. Por ejemplo, si enlazamos $C_1 + C_2$, $C_2 + C_3$ y $C_3 + C_1$ resulta en un explosivo porque se enlazaron 3 pares utilizando 3 compuestos; en cambio, $C_1 + C_2$, $C_2 + C_3$, $C_3 + C_4$ no resulta en un explosivo porque se enlazaron 4 pares utilizando 3 compuestos.

Tu trabajo es consiste en recibir un secuencia de pares enlazados de los compuestos y cargarlos en un camión para su transporte. Pero, si en algún momento recibes un par que puede producir un explosivo, debes de rechazar ese par y continuar con el siguiente par en la secuencia. Por ejemplo, supongamos que recibes la siguientes secuencia de pares:

$(C_1 + C_2, C_7 + C_2, C_4 + C_6, C_1 + C_5, C_5 + C_7, C_6 + C_8)$. En este caso debes de aceptar los primeros cuatro pares pero rechazar el par $C_5 + C_7$ porque se podría hacer el siguiente explosivo $C_1 + C_2, C_7 + C_2, C_1 + C_5, C_5 + C_7$ (4 pares con 4 compuestos). Finalmente, debes de aceptar el último par $C_6 + C_8$.

Construye un algoritmo que toma como entrada una secuencia de pares y retorna el número de pares rechazados.

1. Dado una secuencia S de m pares enlazados de n compuestos, modela el problema como un grafo definiendo formalmente el conjunto de vértices y aristas a partir de S . Escribe por extensión o comprensión los conjuntos de vértices y aristas. ¿Qué representa un bosque como subgrafo de este grafo? ¿Qué representan los ciclos en el grafo?
2. Basándote en tu definición formal del ítem 1, dibuja el grafo para la secuencia $(C_1 + C_2, C_7 + C_2, C_4 + C_6, C_1 + C_5, C_5 + C_7, C_6 + C_8)$.

3. Escribe un algoritmo que toma como entrada una secuencia S de m pares y retorna el número de pares rechazados. Modifica el algoritmo de Kruskal para resolver este problema. Puedes suponer que tienes a tu disposición una estructura de UNION-FIND con las subrutinas de UNION, FIND y MAKESET para la implementación del algoritmo de Kruskal; no necesitas escribir las subrutinas de UNION, FIND o MAKESET. Escribe también una explicación del funcionamiento de tu algoritmo.
4. ¿Cuál es la complejidad temporal de tu algoritmo dado una secuencia S de m pares? Escribe la complejidad utilizando la notación asintótica más ajustada posible. Presenta también una justificación de esa complejidad.

Rúbrica del Ejercicio 2.2

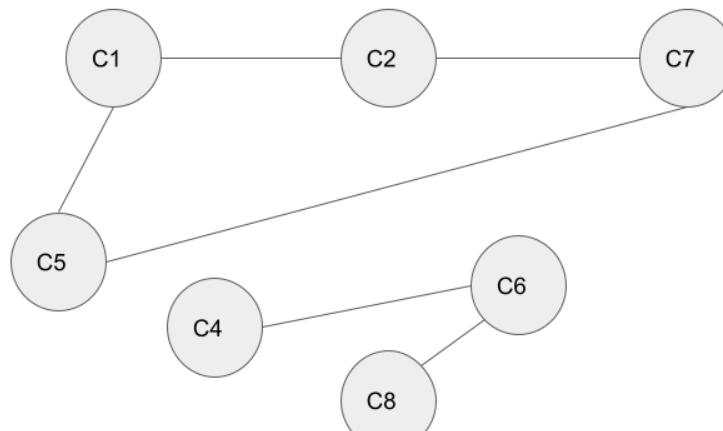
Criterio	1 Punto	2 Puntos
Grafo	Define formalmente y correctamente el conjunto de vértices o aristas	Define formalmente el conjunto de vértices y aristas
Intepretación del grafo	Sobre el grafo definido, presenta una explicación correcta del árbol de expansión mínimo o los ciclos	Sobre el grafo definido, presenta una explicación correcta del árbol de expansión mínimo y los ciclos
Dibujo del grafo	Presenta un dibujo correcto basado en la definición formal	
Algoritmo	Presenta un pseudocódigo que retorna el resultado correcto en la mayoría de las entradas	Presenta un pseudocódigo que retorna el resultado correcto en todas las entradas
Explicación	Escribe una explicación correcta del algoritmo presentado justificando su funcionamiento	
Complejidad	La complejidad asintótica es correcta	La complejidad asintótica es correcta y presenta una justificación correcta de la complejidad

Total 10 puntos

Solución del Ejercicio 2.2

Sea una secuencia de pares $S = (P_1, P_2, \dots, P_m)$ donde cada par $P_i = (C_j, C_k)$ con $i, j \in \{1, 2, \dots, n\}$. A partir de esta secuencia S podemos modelar el problema como un grafo $G = (V, E)$ donde $V = \{C_i : i = 1, 2, \dots, n\}$ y $E = \{C_j C_k : \exists i \in \{1, \dots, m\} \quad C_j, C_k \in P_i\}$. En este grafo, un bosque representa los pares de compuestos en S que no resultan en un explosivo. Un ciclo representa una subsecuencia de pares en S que resultan en un explosivo.

El grafo resultante de la secuencia solicitada se dibuja abajo.



```

//S=[P1,P2,...,Pm] es la secuencia de pares de compuestos
//Cada Pi es un par P[0] y P[1] que guardan los compuestos
COMPUESTOS(S=[P1,P2,...,Pm])
begin
    C = 0 //contador de pares rechazados
    for i=0 to m-1 do
        //si un compuesto no está en ningún conjunto, entonces
        creamos el conjunto
        if FIND(P[i][0]) == NIL then
            MAKESET(P[i][0])
        end-if
        if FIND(P[i][1]) == NIL then
            MAKESET(P[i][1])
        end-if
        //si los pares de compuestos no generan un ciclo,
        entonces los ponemos en un mismo conjunto
        if FIND(P[i][0]) != FIND(P[i][1]) then
            UNION(P[i][0],P[i][1])
        //si los pares de compuestos generan un ciclo,
        incrementamos el contador
        else
            C = C + 1
        end-if
    end-for
    return C
end

```

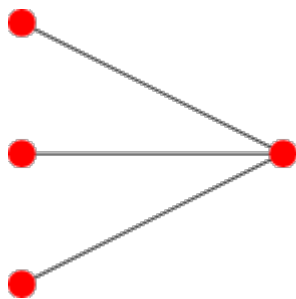
El algoritmo es una modificación del algoritmo de Kruskal, con la diferencia de que solo conoces las aristas del grafo y estos no tienen costos asociados. El algoritmo lee cada par en S , y lo primero que hace cuando lee un par es ver si los compuestos ya están o no en algún conjunto. Si no están, crea la estructura de conjuntos utilizando MAKESET. Esto lo hace para que FIND y UNION puedan funcionar correctamente. El siguiente condicional lo que hace es verificar si dado un par, sus compuestos están o no en un mismo conjunto. Si están en conjuntos diferentes, crea la unión de esos conjuntos para tener un nuevo conjunto que incluya esos compuestos. Si están en un mismo conjunto, al igual que el algoritmo de Kruskal, significa que hay un ciclo entre los

compuestos, por lo tanto ese par debe de rechazarse y se incrementa en 1 el contador de pares rechazados.

La complejidad del algoritmo es m veces la complejidad de FIND, UNION y MAKESET. La complejidad de UNION y MAKESET es $O(1)$ y FIND es $O(\log n)$ porque tenemos n compuestos y son los compuestos los que se utilizan para construir la estructura de árbol de UNION-FIND. Entonces el total es $O(m \log n)$.

Ejercicio 2.3

Una garra es un grafo bipartito $K_{3,1}$ como se muestra en la figura de abajo.



Dado un grafo regular de grado 3, debes de construir un algoritmo que determine si el grafo puede descomponerse en garras. Una descomposición de un grafo G es una lista de subgrafos $(G_1, G_2, \dots, G_k)$ en donde cada arista de G aparece exactamente en un solo grafo G_i de la lista.

1. Dibuja un ejemplo de un grafo con 6 vértices conexo y regular de grado 3 que pueda descomponerse en garras. Escribe por extensión o comprensión los conjuntos de vértices y aristas. Muestra la descomposición en garras.
2. Construye un algoritmo que toma como entrada un grafo conexo y regular de grado 3, y que retorne 1 si el grafo puede descomponerse en una lista $(G_1, G_2, \dots, G_k)$ en donde cada G_i es una garra, y retorna 0 si no puede descomponerse en garras.
3. Escribe una explicación de tu algoritmo justificando su funcionamiento.
4. ¿Cuál es la complejidad temporal de tu algoritmo para un grafo G de n vértices? Escribe la complejidad utilizando la notación asintótica más ajustada posible. Presenta también una justificación de esa complejidad.

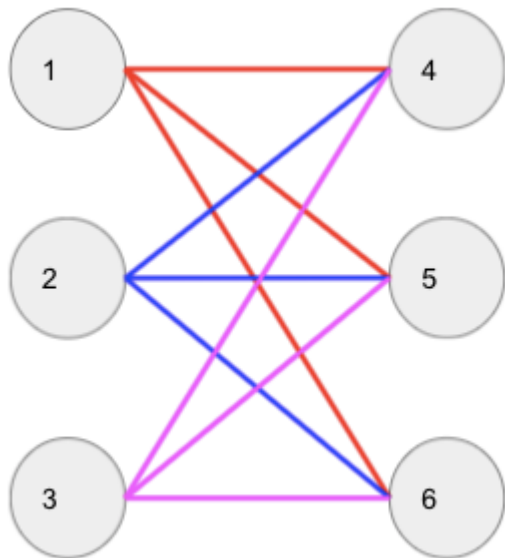
Rúbrica del Ejercicio 2.3

Criterio	1 Punto	2 Puntos
Dibujo del grafo	Presenta un dibujo correcto	
Descomposición	Muestra una descomposición en garras del dibujo dado	
Definición formal	Define formalmente y correctamente el conjunto de vértices o aristas del dibujo	Define formalmente el conjunto de vértices y aristas dibujo

Criterio	1 Punto	2 Puntos
Algoritmo	Presenta un pseudocódigo que retorna el resultado correcto en la mayoría de las entradas	Presenta un pseudocódigo que retorna el resultado correcto en todas las entradas
Explicación	Escribe una explicación del algoritmo presentado justificando su funcionamiento	
Complejidad	La complejidad asintótica es correcta	La complejidad asintótica es correcta y presenta una justificación correcta de la

Solución del Ejercicio 2.3

Abajo se muestra un grafo conexo y regular de grado 3 con 6 vértices. Las 3 garras en que puede descomponerse el grafo están pintados de colores diferentes.



El conjunto de vértices es $V = \{1, 2, 3, 4, 5, 6\}$ y el conjunto de aristas es $E = \{14, 15, 16, 24, 25, 26, 34, 35, 36\}$.

Como el grafo G es 3-regular, para sacar el algoritmo notemos que lo siguiente es cierto: G es bipartito si y solo si G puede descomponerse en garras. Entonces el algoritmo consiste en hacer un recorrido en BFS o DFS para verificar si G es bipartito o no. Si es bipartito, entonces retornamos 1, si no retornamos 0. Abajo mostramos un pseudocódigo basado en BFS.

```
//Siempre suponemos que G es 3-regular, o si no la técnica de
//colorear no funciona
GARRAS( $G=(V, E)$ )
begin
    Q = [u] //inicializamos la cola con un vértice inicial
    //cualquiera de G
    color[u] = 0 //pintamos el vértice u
    Inicializar vector de vértices visitados en 0
    visitados[u] = 1 //se marca u como visitado
    while Q no está vacío do
```

```

    u = Q.dequeue()
    for v adyacente a u do
        if visitados[v] != 1 then
            visitados[v] = 1
            color[v] = 1-color[u] //pintamos del color
opuesto a u
            Q.enqueue(v) //ponemos v en la cola
        else
            if color[v] == color[u] then
                Return 0
            end-if
        end-if
    end-for
end-while
Return 1
end

```

Como el algoritmo es un recorrido BFS de todo el grafo en la entrada, la complejidad temporal es $O(n + m)$, donde n y m son el número de vértices y aristas, respectivamente.

3. Programación Dinámica

Ejercicio 3.1

Dado un vector de enteros V de longitud n , construye un algoritmo basado en programación dinámica para determinar el producto máximo de una subsecuencia contigua en V . Si

$V = [v_1, v_2, \dots, v_n]$, una *subsecuencia contigua* es una subsecuencia $v_{i_1}, v_{i_2}, \dots, v_{i_m}$, con $m \leq n$, donde $i_j = k + j - 1$ para todo $j \in \{1, 2, \dots, m - 1\}$ para algún $k \in \{1, 2, \dots, n\}$. El problema consiste en encontrar una subsecuencia contigua $(v_{i_1}, v_{i_2}, \dots, v_{i_m})$ cuyo producto $\prod_{j=1}^m v_{i_j}$ sea máximo. Por ejemplo, si $V = [2, 3, 4]$, entonces $(2, 3, 4)$ es una subsecuencia contigua y su producto es 24. Si $V = [-2, 0, -3]$, entonces $(-2, 0)$ y $(0, -3)$ son subsecuencias contiguas con producto 0; en este último ejemplo, $(-2, -3)$ es una subsecuencia con producto 6, pero no es contigua.

1. Escribir una formulación de programación dinámica para encontrar el producto máximo de una subsecuencia contigua a partir de un vector de n elementos de números enteros. Identifica los casos bases y la formulación inductiva.
2. Escribe la tabla de memoización para el vector $V = [-2, 0, -3, -2]$ basándote en tu formulación del ítem anterior.
3. Escribe un algoritmo basado en tu formulación que tome un vector V de enteros como entrada y retorne el producto máximo de una subsecuencia contigua en V . Presenta también una explicación del funcionamiento de tu algoritmo.
4. Realiza el análisis de complejidad temporal de tu algoritmo del ítem 3.

Rúbrica del Ejercicio 3.1

Criterio	1 Punto	2 Puntos	3 Puntos
----------	---------	----------	----------

Criterio	1 Punto	2 Puntos	3 Puntos
Formulación	Define un estado apropiado para el problema	Construye el paso inductivo correcto a partir del estado definido o identifica correctamente el caso base	Construye el paso inductivo correcto a partir del estado definido e identifica correctamente el caso base
Tabla de memoización	La tabla solicitada es correcta		
Algoritmo	El algoritmo presentado es correcto en algunas entradas y está basado en la formulación DP	El algoritmo presentado es correcto en todas las entradas y está basado en la formulación DP	
Explicación	Escribe una explicación del algoritmo presentado justificando su funcionamiento		
Complejidad	Presenta un análisis de complejidad correcto		

Solución del Ejercicio 3.1

Este problema es una variante del problema de suma máxima de subarreglos (*maximum subarray*). Por lo tanto, se puede hacer una modificación al algoritmo de Kadane. Por lo tanto, utilizamos la misma formulación de DP para *maximum subarray* pero ligeramente modificado. Sea $P(i)$ el producto máximo de una subsecuencia contigua que termina en la posición i del vector V . Entonces

$$P(i) = \max\{P(i-1) \cdot A[i], 0\}.$$

Notar que aquí consideramos el producto de una subsecuencia vacía, el cual definimos como 0.

La tabla de memoización para $V = [-2, 0, -3, -2]$ es $P = [0, 0, 0, 6]$.

El algoritmo para el producto máximo es el algoritmo de Kadane modificado.

```
//A es un conjunto de números enteros positivos
PRODUCTO-MAXIMO(A=[1...n])
begin
    prod = 1
    resp = 0
    for i=0 to n-1 do
        prod = prod*A[i]
        resp = max(prod,A[i],0)
    end-for
    Return resp
end
```

El algoritmo utiliza el vector A para ir acumulando en cada entrada i el producto máximo acumulado hasta la posición i . Es igual al algoritmo de Kadane, con la diferencia en que se toma el producto en lugar de la suma.

Ejercicio 3.2

Un robot experimental para una competencia de tiro puede acertar a discos que son lanzados a partir de un sistema automatizado. El robot tiene las siguientes restricciones: (1) siempre acierta el primer disco que se lanza, y (2) puede acertar a un disco siempre y cuando su altura no sea mayor que la altura del último disco que a acertado. Por ejemplo, supongamos que el sistema automatizado lanza 5 discos con la siguiente secuencia de alturas (4, 5, 3, 6, 1). En este caso el robot podrá acertar la secuencia (4, 3, 1).

1. Escribir una formulación de programación dinámica para encontrar la longitud de la secuencia más larga posible que el robot puede acertar dado una secuencia de alturas de n discos. Identificar los casos bases y la formulación inductiva.
2. Escribe una tabla de memoización para la secuencia (4, 5, 3, 6, 1).
3. Escribe un algoritmo basado en tu formulación que tome un vector V de enteros positivos como entrada y retorne la longitud solicitada.
4. Realiza el análisis de complejidad temporal de tu algoritmo del ítem 3.

Rúbrica del Ejercicio 3.2

Criterio	1 Punto	2 Puntos	3 Puntos
Formulación	Define un estado apropiado para el problema	Construye el paso inductivo correcto a partir del estado definido o identifica correctamente el caso base	Construye el paso inductivo correcto a partir del estado definido e identifica correctamente el caso base
Tabla de memoización	La tabla solicitada es correcta		
Algoritmo	El algoritmo presentado es correcto en algunas entradas y está basado en la formulación DP	El algoritmo presentado es correcto en todas las entradas y está basado en la formulación DP	
Explicación	Escribe una explicación del algoritmo presentado justificando su funcionamiento		
Complejidad	Presenta un análisis de complejidad correcto		

Puntos 8 puntos

Solución del Ejercicio 3.2

El problema propuesto es exactamente el problema de encontrar una subsecuencia no incremental más larga. Por lo tanto es igual al caso incremental solo que se cambia un símbolo $<$ por un $\geq$ y tenemos que considerar que el primer disco siempre es acertado. Sea $A[1 \dots n]$ un vector que representa la secuencia de alturas de los discos lanzados. Sea $L(j)$ la longitud de una secuencia no incremental más larga que terminan en j . La formulación para $j > 0$ es

$$\begin{aligned}
 L(j) &= 1 + \max\{L(i) : i < j \wedge A[0] \geq A[i] \geq A[j]\} & \text{si } A[j] \leq A[0], \\
 L(j) &= 0 & \text{si } A[j] > A[0],
 \end{aligned}$$

donde el caso base es $L(0) = 1$.

La tabla de memoización para $(4, 5, 3, 6, 1)$ es $L = (1, 0, 2, 0, 3)$.

El se pseudocódigo se muestra a continuación.

```

//A es un conjunto de números enteros positivos
LNIS(A=[1...n])
begin
    L[0] = 1
    for j=1 to n do
        if A[j] > A[0] then
            L[j] = 0
        else
            max = 0
            for i=0 to j-1 do
                if (L[i] <= L[0] and L[i] >= A[j]) and (L[i] >
L[max]) then
                    max = i
                end-if
            end-for
            L[j] = L[max] + 1
        end-if
    end-for
    Return max(L) //retorna el valor máximo de L
end

```

El pseudocódigo lo que hace es implementar directamente la formulación DP, el cual muy parecido para el caso de la subsecuencia incremental más larga. Solo que consideramos que el primer disco es acertado, y por lo tanto, las alturas de los siguientes discos deben de ser menor o igual a esa altura inicial. El objetivo del segundo ciclo es encontrar el $L(i)$ máximo con las restricciones impuestas.

El análisis de complejidad es muy simple porque hay claramente dos bucles anidados, y lo cual da un costo de $O(n^2)$.

Ejercicio 3.3

Sea $A = \{a_1, a_2, \dots, a_n\}$ una colección de números enteros positivos que representan masas de n objetos; por ejemplo, el objeto i tiene masa a_i . Queremos dividir los objetos en dos contenedores de tal forma que la suma de las masas de los objetos en el contenedor 1 sea igual a la suma de las masas en el contenedor 2. Escribe un algoritmo basado en programación dinámica que toma como entrada el conjunto A y retorna 1 si los objetos pueden colocarse en los contenedores bajo la restricción mencionada, y retorna 0 si no es posible colocar los objetos.

1. Explica como este problema puede resolverse como un problema de suma de subconjuntos.
2. Escribe la formulación de programación dinámica para el problema de suma de subconjuntos. Escribe los casos bases y la formulación inductiva.
3. Escribe una tabla de memoización para $A = \{4, 3, 1\}$.
4. Escribe un pseudocódigo para el problema solicitado basado en tu formulación de programación dinámica para suma de subconjuntos. El pseudocódigo debe de incluir un pseudocódigo para el problema de suma de subconjuntos.

Rúbrica del Ejercicio 3.3

Criterio	1 Punto	2 Puntos
Explicación de la reducción	Explica correctamente como el problema solicitado se resuelve como suma de subconjuntos	
Formulación	Escribe la formulación de DP correcta para suma de subconjuntos escribiendo el paso inductivo y los casos bases	
Tabla de memoización	La tabla solicitada es correcta	
Algoritmo	El algoritmo presentado es correcto en algunas entradas y está basado en el formulación DP. Se observa un pseudocódigo para la suma de subconjuntos correcto	El algoritmo presentado es correcto en todas las entradas y está basado en la formulación DP. Se observa un pseudocódigo para la suma de subconjuntos correcto
Explicación de algoritmo	Escribe una explicación del algoritmo presentado justificando su funcionamiento	
Complejidad	Presenta un análisis de complejidad correcto	

Total 7 puntos

Solución del Ejercicio 3.3

Este problema se conoce como el problema de la partición. Dado el conjunto A , queremos saber si existe un subconjunto $S \subseteq A$ tal $\sum S = \sum A \setminus S$, donde $\sum S$ indica la suma de todos los elementos en S y $\sum A \setminus S$ es la suma de todos los elementos en el complemento de S con respecto a A .

Que el conjunto A se pueda dividir en dos partes iguales significa que (i) si $\sum A$ es impar entonces la respuesta es negativa, y (ii) si $\sum A$ es par entonces tenemos que verificar si existe un subconjunto S tal que $\sum S = (\sum A)/2$. El ítem (ii) es el problema de suma de subconjuntos en donde se busca un subconjunto que sume a una cantidad dada.

Recordemos que el problema de suma de subconjuntos se define de la siguiente forma.

Definimos $Q(i, s) = 1$ si existe un subconjunto no vacío de $A_i = \{a_1, \dots, a_i\}$ tal que $\sum A_i = s$, y $Q(i, s) = 0$ si no existe tal subconjunto. Entonces tenemos para el caso base $Q(1, s) = 1$ si $a_1 = s$ y $Q(1, s) = 0$ si $a_1 \neq s$. Para la definición inductiva tenemos $Q(i, s) = Q(i-1, s) \vee Q(i-1, s - a_i)$. Esto nos permite escribir el siguiente algoritmo.

```

//A es un conjunto de números enteros positivos
PARTICION(A=[1...n])
begin
    suma = SUMAR(A) //sumamos todos los elementos en A
    if suma es impar then
        Return 0
    end-if
    //Inicializamos los casos bases
    for s=0 to suma/2 do
        if A[0] == s then
            Q[1,s] = 1
        else
            Q[1,s] = 0
        end-if
    //implementamos la definición inductiva de suma de
    subconjuntos
    for i=2 to n do
        for s=A[i-1] to suma/2 do
            if s-A[i-1] < 0 then
                Q[i,s] = 0
            else
                Q[i,s] = Q[i-1,s] or Q[i-1,s-A[i-1]]
            end-if
        end-for
    end-for
    Return Q[n,suma/2]
end

```

La tabla de memoización del algoritmo es Q . Para el caso de $A = \{1, 3, 4\}$, la tabla de memoización es una matriz de tamaño $(\sum A/2 + 1) \times 3$.

$$Q = \begin{array}{c|ccccc} & 0 & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 1 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 1 \\ 3 & 0 & 0 & 0 & 0 & 1 \end{array}$$

El algoritmo lo que hace reducir el problema de partición al problema de suma de subconjuntos. Primero verifica si la suma total de A es impar o no. Si es par, ejecuta el algoritmo para suma de subconjunto con una suma objetivo igual a la suma total de A dividido 2. El resto del pseudocódigo es la implementación de forma *bottom-up* de la formulación para suma de subconjuntos. Primero se inicializan los casos bases y luego se implementan la definición inductiva. Al final se retorna la última entrada en la tabla de memoización Q que contiene la respuesta.

Como el algoritmo lo que hace es llenar la tabla de memoización, y esta tiene un tamaño de $(\sum A/2 + 1) \times n$, entonces el costo es de $O(n \cdot \max A)$, donde $\max A$ es el elemento de valor máximo en A .

In []: